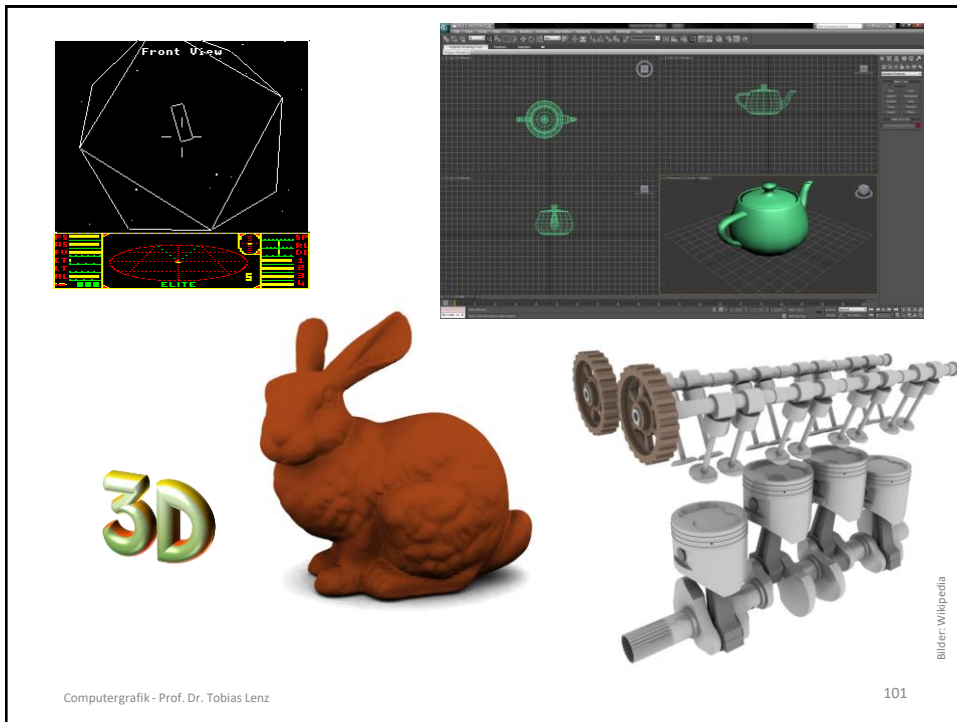
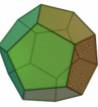




Platonische Körper

- Tetraeder  4 Dreiecke
- Hexaeder  6 Vierecke
- Oktaeder  8 Dreiecke
- Dodekaeder  12 Fünfecke
- Ikosaeder  20 Dreiecke

Grundlagen Upgrade 2D zu 3D

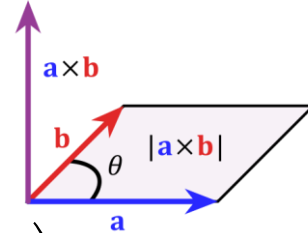
- Linie und Polygon bleiben gleich
 - Folge von Punkten
- Neu: Volumen
 - Ist in 2D immer 0
 - Ohne Bedeutung für uns
- Punkt, Vektor und Matrix bekommen zwei Dimensionen mehr

Skalarprodukt in 3D

- $\vec{a} \cdot \vec{b} = |\vec{a}| |\vec{b}| \cos \varphi$
- Komponentenweise multiplizieren und addieren
- $(a_x, a_y, a_z) \cdot (b_x, b_y, b_z) = a_x b_x + a_y b_y + a_z b_z$
- Interpretation abhängig vom Vorzeichen:
 - > 0 : Vektoren zeigen in selben Halbraum
 - $= 0$: Vektoren stehen senkrecht zueinander
 - < 0 : Vektoren zeigen in unterschiedliche Halbräume

Kreuzprodukt in 3D

- Das Kreuzprodukt der Vektoren a und b ist ein Vektor senkrecht auf a und b (Rechtssystem)
- Länge = Parallelogrammfläche
- $|\vec{a} \times \vec{b}| = |\vec{a}| |\vec{b}| \sin \theta$



$$\begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix} \times \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix} = \begin{pmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{pmatrix}$$

Gespiegelte Welt?

- Matrix von links? Mv
- Oder Matrix von rechts? $v^T M^T$
- Transponieren: Zeilen werden Spalten

$$\begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix}^T = \begin{pmatrix} a & d & g \\ b & e & h \\ c & f & i \end{pmatrix}$$

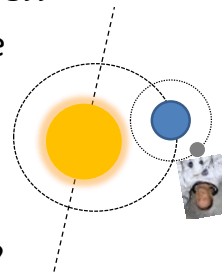
- Schulmathematik/OpenGL nimmt Mv
- DirectX nimmt $v^T M^T$
 - „Natürliche Leserichtung“: $v M_1 M_2 M_3$

Viele Koordinatensysteme

- Hatten schon
 - Fensterkoodinaten in Pixeln
 - Mathematisches Koodinatensystem
- Mit Pixelkoordinaten wollen wir nichts zu tun haben
- Objekte leben im Weltkoordinatensystem
- Modellieren werden wir sie im lokalen Koordinatensystem
- Danach kommen noch weitere...

Hierarchische Koordinatensysteme

- „Weltkoordinaten“: Global
- In Hierarchien: Relative Koordinaten
 - Sonne bewegt sich in der Milchstraße
 - Erde dreht sich um die Sonne
 - Mond dreht sich um die Erde
 - Neil Armstrong auf dem Mond
 - Neils Bewegung in Weltkoordinaten?
- Lokale KS haben eigene Position, Rotation und Skalierung im übergeordneten KS



Homogene Koordinaten: Warum?

- (Ein) Ziel: Jedes lokale KS durch eine Transformation in Weltkoordinaten überführen
- Rotation und Skalierung zusammenfassen
 - $R \cdot S \cdot v = (R \cdot S) \cdot v = M \cdot v$
- 2 Translationen zusammenfassen: $t_2 + t_1 = t$
 - $v + t_2 + t_1 = v + t$
- Rotation und Translation: $R \ ? \ t = ?$
- Viele Transformationen: $R_3(R_2(R_1 v + t_1) + t_2)$
- Ziel: R_3, R_2, R_1, t_1, t_2 zusammenfassen zu M

Homogene Koordinaten: Wie?

- Lösung: Zusätzliche Dimension
- Vektoren haben Form $(x, y, z, 0)$
- Punkte haben Form $(x, y, z, 1)$
 - Allgemein (wx, wy, wz, w) – später wichtig
- Matrizen haben Form

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} & t_1 \\ a_{21} & a_{22} & a_{23} & t_2 \\ a_{31} & a_{32} & a_{33} & t_3 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Translation

- Darstellbar als Matrixmultiplikation

$$\begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x + t_x \\ y + t_y \\ z + t_z \\ 1 \end{pmatrix}$$

Skalierung

- Darstellbar als Matrixmultiplikation

$$\begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} s_x x \\ s_y y \\ s_z z \\ 1 \end{pmatrix}$$

- Gleichförmige Skalierung, falls $s_x = s_y = s_z$
- Sonst: ungleichförmig

Rotation

- x-Achse: $\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha & 0 \\ 0 & \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$
- y-Achse: $\begin{pmatrix} \cos \alpha & 0 & -\sin \alpha & 0 \\ 0 & 1 & 0 & 0 \\ \sin \alpha & 0 & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$
- z-Achse: $\begin{pmatrix} \cos \alpha & -\sin \alpha & 0 & 0 \\ \sin \alpha & \cos \alpha & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Rotation um beliebige Gerade

1. Gerade verschieben, so dass sie durch den Ursprung verläuft
2. Gerade drehen, so dass sie der z-Achse entspricht (2 Drehungen)
3. Drehung um z-Achse ausführen
4. Obige Drehung rückgängig machen
5. Obige Verschiebung rückgängig machen

Einheitsquaternion

- Komplexe mathematische Theorie
 - Brauchen wir aber (fast) alles nicht
- Schreibweisen einer Quaternion
 - Vierdimensionaler Vektor: (w, x, y, z)
 - Skalar + Vektor: $w + v = w + (x, y, z)$
 - Komplexe Zahl: $w + xi + yj + zk$
- Konjugation: $q' = (w, -x, -y, -z)$
- Einheitsquaternion: Länge = 1
 - $w^2 + x^2 + y^2 + z^2 = 1^2$

Rotation per Quaternion

- Multiplikation: $q_1 = w_1 + \vec{v}_1, q_2 = w_2 + \vec{v}_2$
 - $q_1 q_2 = w_1 w_2 - \vec{v}_1 \cdot \vec{v}_2 + w_1 \vec{v}_2 + w_2 \vec{v}_1 + \vec{v}_1 \times \vec{v}_2$
 - Nicht kommutativ wegen Kreuzprodukt
- Rotation eines 3d-Vektors v mittels Quaternion q : qvq'
 - v als Quaternion interpretieren mit $w = 0$
- Quaternion für Drehung um normalisierten Vektor $\vec{v}$ mit Winkel α : $q = \cos \frac{\alpha}{2} + \vec{v} \sin \frac{\alpha}{2}$

Matrix programmieren

- Im Shader gibt es den Typ mat4
- In Java muss eine eigene Klasse programmiert werden (Übung!)
- Typisches Problem
 - Java-Klasse zum Shader übertragen
- Unterschied zu Ecken/Farben
 - Für jede Ecke dieselbe Matrix
 - Also einmalig als “globale Variable” übertragen

Matrix an Shader weitergeben

- Vertex Shader

```
layout(location=0) in vec4 eckenAusJava;
uniform mat4 meineMatrix;
void main() { ...
    gl_Position = meineMatrix * eckenAusJava; }
```
- Java

```
int loc =
glGetUniformLocation(shaderProgram.getId(),
"meineMatrix");
glUniformMatrix4fv(loc,false,matrixFloatArray);
```